LEDGER: Execution Provenance Auditing for Data-Agent Answers

Anonymous Authors

Abstract

Data agents answer questions by selecting sources, writing SQL or Python programs, executing them, and reporting computed values. Auditing these answers requires connecting the stated answer to the final program, declared evidence, and data actually read. We present LEDGER, a deterministic post-execution audit layer that canonicalizes declared sources, reconciles observed reads, replays the final program within a citation-restricted evidence view, and returns a replay certificate, a localized failure, or an unknown status. When replay is unavailable, bounded closure over program-reached operands provides a closure certificate. LEDGER also exposes a task-specific semantic-validation interface, keeping program intent separate from execution provenance. We evaluate these capabilities across three increasingly realistic evidence regimes. On WikiSQL, instantiating the semantic interface with normalized SQL structure detects all 2,715 controlled semantic perturbations, including 466 that preserve the original execution result, with no false positives on 500 unmodified reference queries. On ConvFinQA and MultiHiertt, LEDGER recovers 89.6–93.3% of reference derivations while rejecting 96.1–97.2% of cross-example unsupported answers; standard anomaly detectors reject fewer than 20% under the same high-recall calibration. On human-labelled KramaBench episodes, replay-based selection raises the proportion of delivered answers judged supported from 28.2% to 52.4% while retaining every supported case; on 148 scored live episodes, it increases the mean official score from 0.307 to 0.429 at 64.2% coverage. LEDGER is deterministic, makes zero model calls, and adds sub-millisecond bookkeeping on saved traces. These results show that execution-provenance auditing and task-specific semantic validation provide complementary evidence for selective delivery and escalation.

Keywords: data agents, execution provenance, auditability, selective prediction, semantic validation, data lakes

1. Introduction

Data agents convert natural-language questions into executable data pipelines. They discover sources, inspect schemas, write SQL or Python programs, repair execution errors, and report computed answers with supporting evidence [1, 2, 3]. This workflow changes the object of verification. A reported value may occur in no source file because it is produced by filtering, joining, and aggregation. Auditing a computed answer therefore requires evidence about the execution that produced it, rather than only evidence containing related text or values.

Running example. A KramaBench question asks for the mean number of identity-theft reports among metropolitan areas whose 2023 population exceeds one million. The agent declares 17 CSV files: one national identity-theft table and 16 state-level identity-theft tables. Its final Python program instead reads the national table and 16 state-level fraud-and-other tables. It uses the latter to construct a normalized metropolitan-area name set, infers population as reports divided by the reporting rate per 100,000 times 100,000, retains 14 rows, and returns 150,138/14 = 10,724.1429, exactly the stated answer. However, 16 of the 17 observed reads lie outside the declared evidence view. Restricted replay over the declared files exits normally with `{"main-task": null}`, leaving no output comparable with the stated answer. A different failure remains possible after successful replay: a program may use the wrong aggregation, predicate, or scope while preserving the current output.

These failures induce three audit relations. *Evidence authorization* compares declared evidence with observed host reads. *Execution reproducibility* tests whether the final program reproduces the stated answer within the authorized evidence view. *Semantic correctness* tests whether the program implements the task specification. The first two relations admit deterministic trace and execution witnesses; the third requires a task-specific specification, such as reference SQL, executable constraints, database variants, metamorphic relations, or human judgement. Figure 1 summarizes the three relations and their independent status outputs.

Existing methods address adjacent parts of this chain. Citation metrics evaluate textual support [4, 5]. Database provenance represents how a known query output depends on input tuples [6, 7, 8], while script-provenance systems capture runtime dependencies in controlled environments [9, 10]. Agent benchmarks and model-based judges evaluate final utility or interaction risk [11, 12, 1, 13]. They do not provide a deterministic per-answer witness that jointly connects the final program, stated answer, declared evidence, and observed reads of a completed data-agent episode.

We introduce LEDGER, a deterministic post-execution audit layer for computed data-agent answers. LEDGER freezes the final episode, resolves declared evidence, reconciles host reads, and replays the final program inside the resolved evidence view. It returns a replay certificate, a localized failure, or unknown; when replay is unavailable, bounded closure over program-reached operands provides a derivation witness. A separate task-specific interface produces an independent semantic status. A gover-

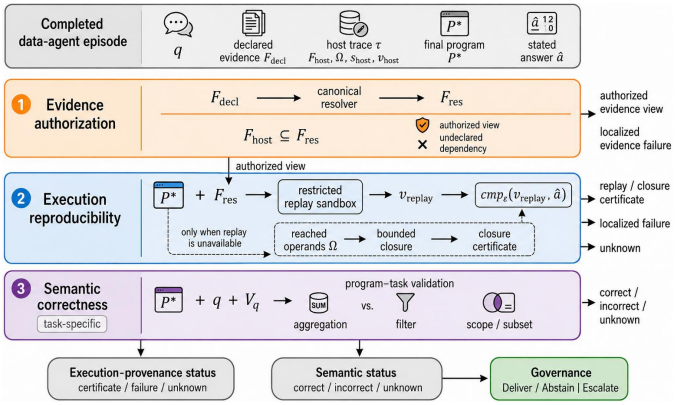


Figure 1: Three relations required to audit a computed data-agent answer. Evidence authorization establishes the permitted evidence view, and execution reproducibility tests whether the final program reproduces the stated answer within that view. Task-specific semantic validation independently evaluates the question–program relation. Governance combines the resulting provenance and semantic statuses.

nance policy then combines the provenance and semantic statuses for delivery, abstention, or escalation.

We make four contributions.

1. **Problem formulation.** We formulate post-execution auditing through evidence authorization, execution reproducibility, and semantic correctness, with an explicit trace contract, trusted computing base, failure semantics, and unknown outcome.
2. **Theory.** We establish the relative soundness of citation-restricted replay, prove completeness of the bounded operand fallback within its operation and depth limits, and formalize why execution-provenance consistency does not imply semantic correctness.
3. **Method.** We develop Ledger, combining canonical evidence resolution, read reconciliation, restricted replay, type-aware output comparison, operand-constrained closure, and an independent task-specific semantic interface. Each outcome carries a machine-readable witness or an explicit unknown status.
4. **Evaluation.** We evaluate the three relations across controlled SQL perturbations, program-labelled numerical reasoning, and live data-agent episodes. Ledger detects same-output program changes, recovers 89.6–93.3% of reference derivations while rejecting 96.1–97.2% of cross-example unsupported answers, and improves the quality of replay-selected live answers at partial coverage.

2. Data-Agent Execution and Audit Setting

2.1. Data-agent environment and completed episode

Environment. A data-agent environment is $\mathcal{E} = (\mathcal{D}, \mathcal{K}, \mathcal{T}, \mathcal{R}_{\text{host}})$, where $\mathcal{D}$ contains typed executable data objects, $\mathcal{K}$ contains schemas, catalog metadata, task instructions, and other planning knowledge, $\mathcal{T}$ is the tool set, and $\mathcal{R}_{\text{host}}$ executes data programs. Any object from $\mathcal{K}$ read by the final program is registered in $\mathcal{D}$ and treated as an execution dependency.

Table 1: Notation for data-agent execution and post-execution auditing.

Symbol	Meaning
Data-agent execution	
$q, \mathcal{A}, h_t, u_t$	Question, data agent, interaction history, and action at turn t.
$\mathcal{E} = (\mathcal{D}, \mathcal{K}, \mathcal{T}, \mathcal{R}_{\text{host}})$	Executable data, planning knowledge, tools, and host runtime.
$\mathcal{I}, \mathcal{L}, \mathcal{Y}$	Evidence-identifier space, program language, and answer domain.
$P^*, F_{\text{decl}}, \hat{a}$	Final program, declared evidence identifiers, and stated answer.
$\text{Exec}_{\mathcal{R}}(P, F)$	Execution returning status, output, observed reads, and reached operands.
τ, E	Frozen host trace and completed data-agent episode.
Post-execution audit	
R, F_{res}	Canonical resolver and resolved declared-evidence view.
$F_{\text{host}}, F_{\text{replay}}$	Reads observed during host execution and restricted replay.
Ω, ρ	Program-reached operands and restricted-replay record.
cmp_ε	Deterministic type-aware output comparator.
$V_q, S(E; V_q)$	Task specification and independent semantic status.
$\text{Audit}(E), c$	Execution-provenance outcome and its failure-code set.

Agent output. A data agent $\mathcal{A}$ answers question q through actions $u_t = \mathcal{A}(q, h_{t-1})$, where h_{t-1} contains earlier actions and tool observations. On termination, it submits $(P^*, F_{\text{decl}}, \hat{a})$: the final executable program $P^* \in \mathcal{L}$, declared evidence identifiers $F_{\text{decl}} \subseteq \mathcal{I}$, and stated answer $\hat{a} \in \mathcal{Y}$. Our live-trace adapters support SQL and Python/pandas; the bounded arithmetic language used without replay is defined in Section 4.3.

Execution record. For runtime $\mathcal{R}$, program P, and mounted data view $F \subseteq \mathcal{D}$, let $\text{Exec}_{\mathcal{R}}(P, F) = (s, v, F_{\text{read}}, \Omega)$ return execution status s, output $v \in \mathcal{Y} \cup \{\perp\}$, observed reads F_{read}, and the multiset Ω of scalar values reaching the final computation. Here $\perp$ denotes the absence of a comparable output. The final host execution is $\text{Exec}_{\mathcal{R}_{\text{host}}}(P^*, \mathcal{D}) = (s_{\text{host}}, v_{\text{host}}, F_{\text{host}}, \Omega)$. We freeze $\tau = (P^*, F_{\text{decl}}, F_{\text{host}}, \Omega, s_{\text{host}}, v_{\text{host}})$ and define the completed episode as $E = (q, \mathcal{E}, \tau, \hat{a})$. The full log may additionally retain program revisions, tool calls, exceptions, and timestamps.

2.2. Audit task and trust model

Resolved view and replay. The canonical resolver $R : \mathcal{I} \to \mathcal{D} \cup \{\perp\}$ maps each declared identifier to a registered data object. The resolved view is $F_{\text{res}} = \{R(f) : f \in F_{\text{decl}}, R(f) \neq \perp\}$ and is complete when every declaration resolves. Restricted replay produces $\rho = \text{Exec}_{\mathcal{R}_{\text{replay}}}(P^*, F_{\text{res}}) = (s_{\text{replay}}, v_{\text{replay}}, F_{\text{replay}}, \Omega_{\text{replay}})$.

Audit outputs. A fixed comparator $\text{cmp}_\varepsilon : \mathcal{Y} \times \mathcal{Y} \to \{1, 0, \perp\}$ returns agreement, mismatch, or non-comparability; numeric outputs use tolerance ε, and structured outputs use deterministic normalization. The execution-provenance audit returns $\text{Audit}(E) \in \{\text{cert}_{\text{replay}}, \text{cert}_{\text{closure}}, \text{failure}(c), \text{unknown}\}$, where a certificate carries an execution or expression witness, c stores one or more observed failure codes and their trace evidence, and unknown denotes that neither a certificate nor an explicit failure is established.

Semantic validation independently returns $S(E; V_q) \in \{\text{correct}, \text{incorrect}, \text{unknown}\}$ for a task-specific specification V_q, such as reference SQL, executable constraints, database variants, metamorphic relations, or human judgement. The provenance status evaluates the program–evidence–answer relation; the semantic status evaluates the question–program relation.

A completed episode induces evidence authorization $F_{\text{host}} \leftrightarrow F_{\text{res}}$, execution reproducibility $(P^*, F_{\text{res}}) \leftrightarrow \hat{a}$, and semantic correctness $(q, V_q) \leftrightarrow P^*$. The first checks whether observed host reads remain within the resolved declared view; the second checks whether P^* reproduces $\hat{a}$ in that view; the third checks whether P^* implements the task specification. Section 3 gives their certificate conditions and guarantees.

Trusted components. The host agent, generated program, declared evidence, host execution, and stated answer are audited objects. LEDGER trusts the trace logger, canonical resolver, restricted replay runtime, and type-aware comparator. A certificate is relative to the integrity and coverage of these components; tampered logs, sandbox violations, external nondeterminism, or uninstrumented dependencies produce unknown or a platform-level failure rather than a certificate.

Running example. In the identity-theft episode, all 17 declarations resolve, so $|F_{\text{decl}}| = |F_{\text{res}}| = 17$. A reconstruction from the preserved final program, confirmed by a fresh trace, gives $|F_{\text{host}}| = 17$ but $|F_{\text{host}} \cap F_{\text{res}}| = 1$; hence $F_{\text{host}} \not\subseteq F_{\text{res}}$. Host execution succeeds with $v_{\text{host}} = \hat{a} =$ 10,724.1429. Restricted replay also exits normally, but returns `{"main-task": null}`, represented as $v_{\text{replay}} = \perp$ and $\text{cmp}_\varepsilon(v_{\text{replay}}, \hat{a}) = \perp$. Thus $\mathsf{Audit}(E) = \mathsf{failure}(c)$ with $c =$ {undeclared host read, missing comparable output}; no replay certificate is issued.

3. Audit Relations and Guarantees

Section 2 defines the completed episode and its audit interfaces. We now formalize the three audit relations summarized in Figure 1. Evidence authorization establishes the data view permitted by the agent's declaration. Execution reproducibility determines whether the final program reproduces the stated answer within that view. Semantic correctness independently evaluates whether the program implements the requested task. The first two relations determine the execution-provenance status; the third determines the semantic status.

3.1. Evidence authorization

The declared evidence first determines the data view against which the recorded execution is audited.

Definition 1 (Evidence authorization)**.** *A completed episode E is evidence-authorized when every declared evidence identifier resolves and*

$$F_{\text{host}} \subseteq F_{\text{res}}, \qquad F_{\text{res}} = R(F_{\text{decl}}).$$

The two conditions produce distinct audit evidence. An unresolved declaration identifies a claimed source that cannot be included in the evidence view. An undeclared host read identifies an observed execution dependency outside that view. Canonical resolution precedes the subset test so that aliases and path variants refer to the same data object.

Evidence authorization establishes F_{res} as the input view for restricted replay. It does not yet establish that the final program reproduces the stated answer.The running example violates this relation because 16 fraud-and-other files read by P^* are absent from F_{res}.

3.2. Execution reproducibility

Let

$$\rho = \text{Exec}_{\mathcal{R}_{\text{replay}}}(P^*, F_{\text{res}}) = (s_{\text{replay}}, v_{\text{replay}}, F_{\text{replay}}, \Omega_{\text{replay}})$$

be the restricted replay record defined in Section 2.2. A replay certificate is issued exactly when

$$\begin{aligned} \text{Cert}_{\text{replay}}(E) = 1 \iff\ & F_{\text{res}} = R(F_{\text{decl}}) \text{ is complete}, \\ & F_{\text{host}} \subseteq F_{\text{res}}, \quad s_{\text{host}} = \text{success}, \\ & \rho = \text{Exec}_{\mathcal{R}_{\text{replay}}}(P^*, F_{\text{res}}), \\ & s_{\text{replay}} = \text{success}, \quad F_{\text{replay}} \subseteq F_{\text{res}}, \\ & \text{cmp}_\varepsilon(v_{\text{replay}}, \hat{a}) = 1. \end{aligned} \tag{1}$$

The replay witness stores the final program snapshot, resolved evidence identifiers, host and replay reads, replay output, comparator, and tolerance.

Theorem 1 (Relative soundness of replay)**.** *Under the trusted execution model of Section 2.2, a replay certificate establishes that P^* reproduces $\hat{a}$ within the resolved declared evidence view, and that all observed host and replay data reads remain inside that view.*

Proof sketch. Equation 1 requires complete evidence resolution, authorized host reads, restricted replay, authorized replay reads, and output agreement. The trusted logger, resolver, replay runtime, and comparator make these conditions observable and enforceable. The full proof appears in Appendix Appendix A.

The guarantee is relative to the recorded final program and the resolved evidence view. It establishes how the stated answer was reproduced, while leaving the question–program relation to the semantic audit. Its restricted replay additionally returns a null result, so $\text{cmp}_\varepsilon(v_{\text{replay}}, \hat{a}) = \perp$.

3.3. Semantic correctness

Semantic correctness evaluates whether P^* implements the task-specific specification V_q for question q:

$$S(E; V_q) \in \{\text{correct}, \text{incorrect}, \text{unknown}\}.$$

Depending on the task, this relation may inspect the requested aggregation, predicates, selected scope, output type, or other executable constraints. It is evaluated independently of whether the recorded program reproduces the stated answer.

Proposition 1 (Semantic non-implication)**.** *There exist completed episodes for which*

$$\text{Cert}_{\text{replay}}(E) = 1 \qquad \textit{and} \qquad S(E; V_q) = \text{incorrect}.$$

Consequently, execution-provenance consistency does not imply semantic correctness.

Proof sketch. Choose two programs with different task semantics but identical outputs on the current data. Replay certifies the submitted program's output, while the task-specific validator rejects its operation. Appendix Appendix A gives the full argument.

For example, a sum and an average coincide after filtering to one row, and an incorrect predicate may leave the result unchanged on the current database. Such episodes are reproducible but violate the task specification. The same separation applies to the bounded closure introduced in Section 4.3: derivability from reached operands does not identify the operation requested by the question. This distinction motivates the independent semantic branch in Figure 1 and the same-output SQL evaluation in Section 5.4.

4. The LEDGER Audit Layer

Overview. Given a completed episode E, LEDGER resolves its declared evidence, reconciles observed reads, replays the final program in the resolved view, and returns $\mathsf{Audit}(E)$ with a machine-readable witness or failure record. When replay is unavailable, bounded closure over Ω provides a fallback. A task-specific interface independently produces $S(E; V_q)$; the two statuses meet only at governance.

4.1. Trace capture and evidence normalization

Trace capture. The host wrapper freezes the final program, declared evidence, execution status and output, observed data reads, exceptions, accessed attributes, and operands reaching the returned result. Python/pandas and SQL adapters map these records into the common trace τ defined in Section 2.1. Declared sources, host reads, and replay reads remain separate fields.

Evidence normalization. The resolver maps declarations to $F_{\text{res}} = R(F_{\text{decl}})$ before comparing F_{host} with the resolved view. This step implements evidence authorization and supplies the data view used by restricted replay.

4.2. Episode audit and restricted replay

Audit order. Algorithm 1 resolves evidence and checks host-read authorization before replay, then records replay-boundary, execution, and output-comparison conditions. A failure record c may retain several observed conditions; closure is considered only when replay is unavailable and no observed failure already determines the outcome.

Replay witness. The replay runtime mounts only F_{res}, records F_{replay}, and applies $\text{cmp}_\varepsilon(v_{\text{replay}}, \hat{a})$. Deterministic normalization handles strings, booleans, collections, and structured outputs; numeric values use tolerance ε. A replay certificate stores P^*, F_{res}, host and replay reads, replay output, and comparator configuration.

Running example. For the identity-theft episode, all 17 declared files resolve, so $|F_{\text{decl}}| = |F_{\text{res}}| = 17$. The preserved final program P^* reads the national identity-theft table and 16 state-level fraud-and-other tables. Because the archived episode did not retain syscall reads, we reconstruct F_{host} from the exact program and verify it with a fresh trace: $|F_{\text{host}}| = 17$ but $|F_{\text{host}} \cap F_{\text{res}}| = 1$, leaving 16 undeclared reads.

The program unions 87 metropolitan-area names from those 16 files, filters the national table, infers population as $\text{pop} = \text{reports}/\text{rate}_{100\text{k}} \times 100{,}000$, and retains 14 rows. Their report sum is 150,138, yielding $v_{\text{host}} = 10{,}724.1429$ after rounding; this equals $\hat{a}$.

Restricted replay mounts only F_{res}: the 16 fraud-file existence checks fail, the valid-name set is empty, and the program exits normally with `{"main-task": null}`. Thus $s_{\text{replay}} = \text{success}$, $F_{\text{replay}} \subseteq F_{\text{res}}$, and $\text{cmp}_\varepsilon(v_{\text{replay}}, \hat{a}) = \perp$. The failure record therefore contains both an undeclared-host-read witness and a missing comparable replay output; no replay certificate is issued.

Algorithm 1 LEDGER episode audit

Require: episode E, resolver R, replay runtime $\mathcal{R}_{\text{replay}}$, comparator cmp_ε, operations $\mathcal{O}$, depth d

$c \leftarrow [\,]$; $b \leftarrow \perp$
$F_{\text{res}} \leftarrow R(F_{\text{decl}})$
if F_{res} is incomplete **then**
  append unresolved source to c
else
  if $F_{\text{host}} \not\subseteq F_{\text{res}}$ **then**
    append undeclared host read to c
  end if
end if
if $s_{\text{host}} \neq$ success **then**
  append host execution to c
end if
if F_{res} is complete and P^* is executable in $\mathcal{R}_{\text{replay}}$ **then**
  $\rho \leftarrow \text{Exec}_{\mathcal{R}_{\text{replay}}}(P^*, F_{\text{res}})$
  if $F_{\text{replay}} \not\subseteq F_{\text{res}}$ **then**
    append replay boundary to c
  end if
  if $s_{\text{replay}} \neq$ success **then**
    append replay execution to c
  else
    $b \leftarrow \text{cmp}_\varepsilon(v_{\text{replay}}, \hat{a})$
    if $b = 0$ **then**
      append output mismatch to c
    else if $b = \perp$ **then**
      append missing comparable output to c
    end if
  end if
  if $c = [\,]$ and $b = 1$ **then**
    return $\mathsf{cert}_{\text{replay}}$ and replay witness
  end if
end if
if $c \neq [\,]$ **then**
  return $\mathsf{failure}(c)$ and diagnostic record
end if
if P^* is not executable in $\mathcal{R}_{\text{replay}}$ and Ω is complete **then**
  return BOUNDEDCLOSURE$(\Omega, \mathcal{O}, d, \hat{a})$
end if
return unknown

4.3. Operand-constrained fallback

Fallback witness. When a runnable replay environment is unavailable, LEDGER searches only over the program-reached

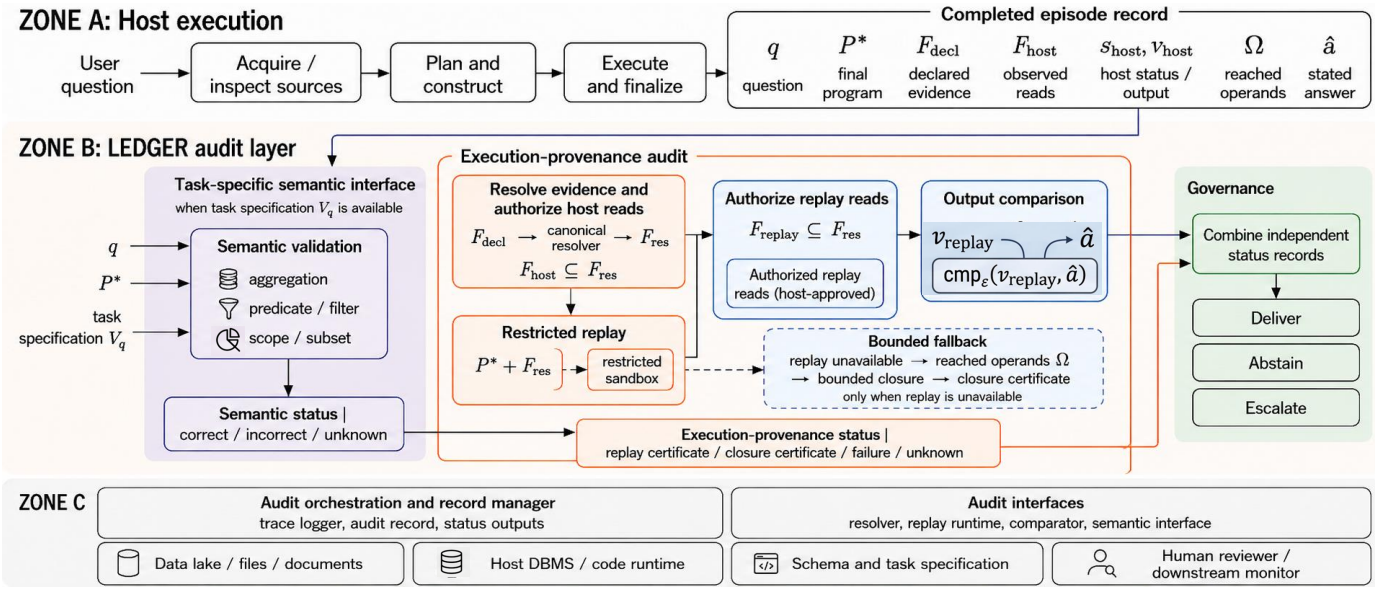


Figure 2: Architecture of the LEDGER audit layer. A completed episode is copied to independent execution-provenance and task-specific semantic branches. The provenance branch resolves evidence, reconciles host and replay reads, performs restricted replay, and invokes bounded closure only when replay is unavailable. Governance combines the two status records.

Algorithm 2 Bounded operand closure

function BOUNDEDCLOSURE($\Omega, O, d, \hat{a}$)
  $S_0 \leftarrow$ normalize(Ω)
  for $j = 1, \ldots, d$ **do**
    $S_j \leftarrow S_{j-1}$
    for all $x, y \in S_{j-1}$ and $o \in O$ **do**
      add valid normalized $o(x, y)$ with an expression witness
    end for
  end for
  if $\hat{a} \in S_d$ within ε **then**
    return $\mathsf{cert}_{\mathrm{closure}}$ and a minimal witness
  end if
  return unknown
end function

operands rather than all numbers in the cited context. With $O = \{\mathrm{sum}, \mathrm{diff}, \mathrm{ratio}, \mathrm{pctchg}, \mathrm{prod}\}$, the depth-$d$ closure recursively applies valid operations, normalizes results, and removes duplicates.

Definition 2 (Closure certificate)**.** *Episode E receives a depth-d closure certificate when $\hat{a} \in \mathrm{Cl}_d(\Omega)$ within tolerance ε. The witness stores a minimal expression and its operand subset.*

Proposition 2 (Closure completeness)**.** *If $\hat{a}$ is generated from reached operands by operations in O, expression depth at most d, and the audit's normalization and tolerance, then $\hat{a} \in \mathrm{Cl}_d(\Omega)$.*

Proof sketch. The operands form S_0. Induction on expression depth places every valid internal result in the next closure level, so the root appears by depth d. The full proof appears in Appendix A.

Table 2: Interface between audit statuses and governance actions.

Provenance status	Semantic status	Action
certificate	correct	deliver
certificate	unknown	deliver or review
certificate	incorrect	abstain / escalate
failure or unknown	any	abstain / escalate

A closure certificate establishes derivability from Ω; it does not carry the read-set guarantee of a replay certificate or identify the operation requested by q.

4.4. Semantic interface and governance

Semantic status. The semantic interface consumes q, P^*, and the task specification V_q and returns $S(E; V_q)$. SQL tasks may compare normalized program structure or execute programs on database variants [14]; other tasks may provide executable constraints, metamorphic relations, or human judgement. This status remains separate from $\mathsf{Audit}(E)$.

Governance. A policy maps the two statuses to deliver, abstain, or escalate. A provenance certificate with incorrect semantics is withheld, while a failure or unknown provenance status is abstained from or escalated. A certificate with unknown semantic status is handled by deployment policy.

4.5. Complexity and deterministic contract

Cost and determinism. Resolution and read reconciliation cost $O(|F_{\mathrm{decl}}| + |F_{\mathrm{host}}|)$, and replay adds one execution of P^*. If $n_j = |S_j|$ and $r = |O|$, closure level j generates at most rn_{j-1}^2 candidates before normalization and duplicate removal. For a fixed trace, resolver, replay runtime, comparator, operation vocabulary, and depth, LEDGER is deterministic and invokes no language model.

Table 3: Evaluation layers and their primary roles. Size reports total/evaluable instances where the distinction applies.

Layer	Dataset	Size	Primary use
Closure	ConvFinQA	388/386	Arithmetic audit and baselines
Closure	MultiHiertt	7,830/6,301	Transfer and depth stress
Semantic	WikiSQL	500/2,715	Same-output program changes
Live audit	KramaBench	166	Replay and selective delivery
Live audit	DABstep	60	Completed-episode transfer
Human	KramaBench	40/39	Deliverability and escalation

5. Evaluation

5.1. Questions, evidence layers, and metrics

We ask four questions.

RQ1 How does operand-constrained auditing compare with ordinary numeric, trace-rule, anomaly, and model-based checks?

RQ2 Does the frozen audit transfer across datasets, and how does closure depth expose its derivation–collision trade-off?

RQ3 Which semantic changes remain invisible to execution-result equality, and which complementary checks detect them?

RQ4 Do replay certificates improve the quality of completed episodes delivered to users or downstream systems?

Evidence layers. ConvFinQA and MultiHiertt evaluate the bounded-closure component using program-labelled numerical derivations and aligned cross-example answer substitutions. WikiSQL evaluates the task-specific semantic interface through controlled program perturbations. KramaBench and DABstep evaluate completed data-agent episodes through restricted replay, failure localization, and selective delivery. The KramaBench human label is a composite deliverability judgement: it asks whether the stated answer follows from the cited data through the recorded computation and addresses the requested quantity.

Metrics. For program-labelled examples, *reference-derivation recall* is the fraction of supported derivations that receive a closure certificate, and *cross-example rejection* is the fraction of aligned unsupported substitutions that do not receive one. For WikiSQL, *perturbation detection* is the fraction of modified queries identified as changed, while the false-positive rate is measured on unmodified reference queries. For selective delivery, *coverage* is the fraction of evaluable episodes delivered; *delivered support* is the fraction of delivered episodes judged supported by human annotators; *supported-case retention* is the fraction of all human-supported episodes retained; and *mean delivered score* averages the benchmark's official score over delivered episodes.

Baselines. Numeric baselines include exact operand matching, one-operation derivability, and closure over all numbers in the cited context. Trace rules check execution success, citation resolution, declared-read coverage, and unrestricted replay. Anomaly baselines use robust MAD distance, Isolation Forest [15], Local Outlier Factor [16], and one-class SVM [17]. Their thresholds use only the 60-example calibration seed and retain at least 90% supported derivations on that seed. A supervised logistic model provides an upper baseline. No anomaly feature contains the audit verdict or benchmark answer.

Table 4: ConvFinQA baselines on the held-out remainder.

Method	Derivation recall	Cross-example rejection
Single operation	276/386 (0.715)	383/386 (0.992)
All cited numbers	332/386 (0.860)	240/386 (0.622)
Isolation Forest	351/388 (0.905)	74/388 (0.191)
Supervised upper	349/388 (0.899)	230/388 (0.593)
LEDGER	**360/386 (0.933)**	**371/386 (0.961)**

Evidence-aware judge. A local Qwen3.6-35B judge receives the question, declared evidence metadata, observed reads, program summary, execution status, recomputed output, and stated answer. It receives neither benchmark gold nor LEDGER labels. Three temperature-zero repeats measure response stability. Transferred human records omit most raw rows and complete program bodies, so that subset remains a limited-evidence diagnostic.

5.2. RQ1: Program-reached operands separate derivations from numeric coincidence

Trace-local closure preserves derivations and suppresses unrelated numbers. ConvFinQA supplies 386 evaluable reference derivations and aligned cross-example substitutions; MultiHiertt supplies 6,301 aligned pairs. The frozen checker uses program-reached operands, five operations, depth 2, and relative tolerance 10^{-4}. It recovers 93.3% and 89.6% of the reference derivations on ConvFinQA and MultiHiertt, while rejecting 96.1% and 97.2% of the corresponding cross-example substitutions. Closure over all cited-context numbers recovers additional derivations over one-operation checking, but also admits many unrelated numeric coincidences.

Standard anomaly scores do not produce a derivation witness. Isolation Forest, Local Outlier Factor, and one-class SVM retain most supported examples but reject only 13.7–19.8% of the cross-example substitutions. Their trace features identify unusual executions, but do not establish that the answer follows from the program-reached values. Restricting closure to those operands preserves the relevant search space and removes unrelated context numbers before recursive expansion.

5.3. RQ2: Closure depth recovers derivations and expands the collision surface

Depth 2 captures short programs while retaining cross-example rejection. We stratify MultiHiertt by reference-program depth and vary closure depth on the disjoint ConvFinQA calibration seed. Most remaining MultiHiertt misses correspond to programs deeper than the frozen bound: 72.3% of misses exceed depth 2. Increasing calibration depth from 1 to 3 recovers every supported seed derivation, while cross-example rejection falls sharply (Figure 4a,b).

Additional depth enlarges the semantic collision surface. We replace each ConvFinQA answer with the result of a different operation over the same operands. Across all 382 substitutions, only 9 are rejected; none of the 347 substitutions within the

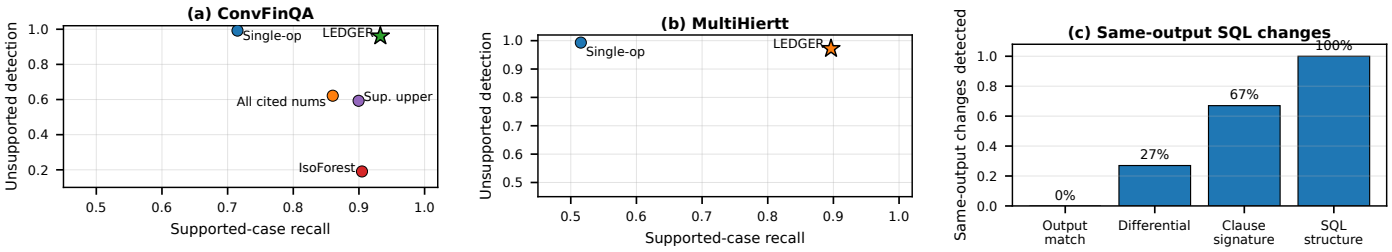


Figure 3: Operand-constrained auditing on ConvFinQA and MultiHiertt, together with the semantic complement on WikiSQL. Panels (a,b) report reference-derivation recall and cross-example rejection; panel (c) reports detection on same-output SQL changes.

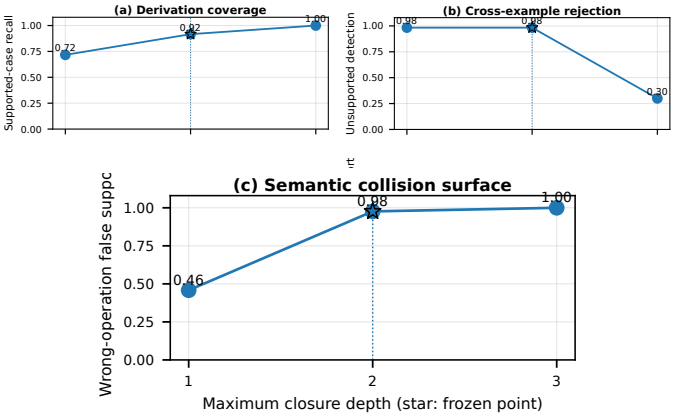


Figure 4: Effect of closure depth on derivation coverage, cross-example rejection, and wrong-operation false support. The star marks the frozen depth-2 setting.

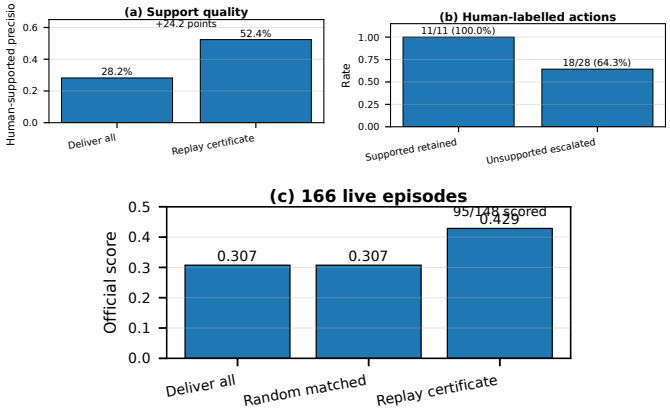


Figure 5: Replay-based selection improves delivered support, retains human-supported episodes, escalates unsupported episodes, and raises mean delivered score at partial coverage.

frozen operation vocabulary and depth is rejected. Deeper closure accepts still more variants because the altered answers remain derivable from the same values (Figure 4c). Closure therefore records arithmetic derivability, while operation identity remains a semantic property.

5.4. RQ3: Structural validation detects same-output program changes

Execution-result equality misses semantic changes. From 500 executable WikiSQL development examples, we construct 2,715 perturbations that alter aggregation, projection, predicate column or value, or predicate presence. Execution-result equality misses 466 perturbations because the modified and reference programs return the same result on the original database.

Program structure complements replay. Normalized SQL structure detects all 2,715 perturbations with no false positives on the 500 unmodified reference queries. On a 100-case same-output sample, operator/clause signatures detect 67 changes and differential execution detects 27, while original-database output comparison detects none (Figure 3c). Structural comparison observes the changed program directly; execution on database variants succeeds when the variants separate the candidate and reference programs. SQL deployments can therefore instantiate V_q through structural and test-suite methods [14]; model-based metrics such as FLEX remain an alternative when reference programs are unavailable [18].

5.5. RQ4: Replay certificates support selective delivery

Replay status provides a human-facing delivery signal. The live corpus contains 166 KramaBench and 60 DABstep episodes. Among 39 KramaBench episodes with clear human labels, delivering only replay-certified episodes retains all 11 supported cases and escalates 18 of 28 unsupported cases. The proportion of delivered answers judged supported consequently rises from 28.2% to 52.4% (Figure 5a,b). The remaining replay-certified but human-unsupported episodes reproduce a value through programs that address the wrong quantity, scope, or operation, consistent with the separation between provenance and semantics.

The same certificate improves score at partial coverage. Among 148 KramaBench episodes with finite official scores, replay-certificate delivery selects 95 episodes, corresponding to 64.2% coverage. Their mean official score is 0.429, compared with 0.307 for delivering all episodes and 0.307 for uniformly random selection at the same size (Figure 5c). The policy changes which completed episodes are delivered; it does not modify host-agent source selection, program generation, or execution.

5.6. Runtime, determinism, and model baseline

Deterministic bookkeeping is small relative to host execution. Across 500 saved records, the audit adds sub-millisecond median bookkeeping and makes zero model calls. Five independent

repeats over 100 records produce identical execution-provenance outcomes. These measurements cover record parsing, comparison, and closure over saved traces; they exclude host-agent execution and external data loading.

The model judge remains a diagnostic baseline. An evidence-aware 35B judge shows label disagreement, structured-output failures, and occasional repeat flips. The complete counts appear in Appendix Appendix B. Judge outputs do not alter the deterministic execution-provenance status.

6. Related Work

Data-agent evaluation.. KramaBench evaluates data-to-insight pipelines over heterogeneous lake files [1]; DABstep targets multi-step tool-based analysis [2]. LakeQA scales discovery and reasoning to a million-scale lake [3]. SANA decomposes failures across search, planning, analysis, and policy [13]. These benchmarks motivate process-level evidence in addition to final-answer accuracy.

Execution provenance and auditability.. Database provenance formalizes how outputs depend on inputs [6, 7, 8]. noWorkflow and Vamsa capture dependencies in data-science scripts [9, 10]. Agent-trace work organizes retrieved evidence, tool outputs, memory, actions, and final claims under process-level accountability [19]. Auditable Agents distinguishes accountability, auditability, and auditing and identifies action recoverability and evidence integrity as core dimensions [20]. Ledger operationalizes these dimensions for completed computed answers and their declared evidence view.

Agent governance, security, and trace evaluation.. Contextual security frameworks separate task alignment, action alignment, source authorization, and data isolation [21]; intent-to-execution integrity further separates tool, instruction, judgment, and data-flow properties [22]. The provenance status in Ledger covers an observed source and execution boundary, while its task-specific semantic interface addresses a separate alignment relation. TrustAgent intervenes before, during, and after planning through an agent constitution [23]; Ledger instead audits a completed episode. ToolEmu, AgentDojo, and R-Judge establish tool trajectories as standard evaluation objects [11, 12, 24].

Citation, semantic, and selective evaluation.. ALCE and FActScore evaluate textual citation support and factual claims [4, 5]. Text-to-SQL test suites approximate semantic equivalence through execution on multiple database variants [14]; FLEX uses an LLM-based expert metric to reduce execution-metric errors [18]. Selective QA trades coverage for reliability using confidence [25]. Ledger derives its delivery signal from a replay witness and retains task-specific semantic evaluation as an independent control.

7. Conclusion

Computed data-agent answers require evidence that spans declared sources, observed reads, final programs, execution outcomes, and stated values. Ledger converts a completed episode into a deterministic execution-provenance record through source resolution, read reconciliation, citation-restricted replay, and bounded operand closure. Ordinary rules and anomaly detectors do not match its joint derivation recall and unsupported-answer detection. Live traces show that replay certificates support selective delivery while preserving an explicit unknown path. Controlled SQL perturbations establish the complementary role of task-specific semantic validation. Together, the two independent statuses provide a reproducible basis for delivery and escalation decisions.

Appendix A. Formal Guarantees and Proofs

Appendix A.1. Proof setup

We first make explicit the assumptions used by the guarantees. They correspond to the trusted components defined in Section 2.2.

(A1) Trace fidelity. The frozen snapshot P^* is the program submitted by the host agent, and F_{host} contains every data-object read observed during its final execution.

(A2) Resolution consistency. The resolver R maps each resolvable evidence identifier to one canonical data object. A resolved view is complete exactly when every element of F_{decl} is mapped.

(A3) Replay confinement. The replay runtime exposes only F_{res}, records every data-object read in F_{replay}, and returns the recorded status and output of P^* in that view.

(A4) Comparator consistency. For supported output types, $\text{cmp}_\varepsilon(x, y) = 1$ exactly when the deterministic normalization and tolerance rules treat x and y as equal.

Write $x \simeq_\varepsilon y$ when $\text{cmp}_\varepsilon(x, y) = 1$. Define evidence authorization as

$$\text{Auth}(E) \equiv [F_{\text{res}} = R(F_{\text{decl}}) \text{ is complete}] \wedge [F_{\text{host}} \subseteq F_{\text{res}}].$$

For the replay record $\rho = (s_{\text{replay}}, v_{\text{replay}}, F_{\text{replay}}, \Omega_{\text{replay}})$, define

$$\begin{aligned}\text{Rep}(E) \equiv\ & s_{\text{host}} = \text{success} \wedge s_{\text{replay}} = \text{success} \\ & \wedge\ F_{\text{replay}} \subseteq F_{\text{res}} \wedge v_{\text{replay}} \simeq_\varepsilon \hat{a}.\end{aligned}$$

Equation 1 can then be written compactly as $\text{Cert}_{\text{replay}}(E) = 1$ if and only if $\text{Auth}(E) \wedge \text{Rep}(E)$.

Appendix A.2. Proof of Theorem 1

Proof. Suppose Algorithm 1 returns $\mathsf{cert}_{\text{replay}}$. The algorithm returns this outcome only when its failure record is empty and $\text{cmp}_\varepsilon(v_{\text{replay}}, \hat{a}) = 1$.

An empty failure record implies that all declared identifiers resolve, $F_{\text{host}} \subseteq F_{\text{res}}$, host execution succeeds, restricted replay succeeds, and $F_{\text{replay}} \subseteq F_{\text{res}}$. Hence $\text{Auth}(E)$ holds. Assumptions (A1) and (A2) make the first read-set inclusion a statement about the recorded host execution rather than identifier aliases.

By (A3), the replay execution of P^* is performed with only F_{res} exposed. The replay status is successful and all observed replay reads remain inside that view. By (A4), $\text{cmp}_\varepsilon(v_{\text{replay}}, \hat{a}) =$

1 implies $v_{\text{replay}} \simeq_\varepsilon \hat{a}$. Therefore Rep($E$) holds. It follows that P^* reproduces the stated answer within the resolved declared evidence view, and that every observed host and replay data read belongs to that view.

Conversely, consider any failed conjunct in Equation 1. Incomplete resolution, an unauthorized host or replay read, an execution failure, an output mismatch, or an incomparable output either adds a corresponding condition to the failure record or prevents the comparator result from equalling one. Algorithm 1 therefore cannot return $\mathsf{cert}_{\text{replay}}$. □

Interpretation.. The theorem certifies a concrete program–evidence–answer link: the recorded program reproduces the stated answer using the resolved declared evidence, under the trusted execution model. The theorem does not compare P^* with the task specification; that relation is addressed by Proposition 1.

Appendix A.3. Proof of Proposition 2

Let $\mathsf{Expr}(\Omega, O)$ be the expression language generated by

$$e ::= \omega \mid o(e_1, e_2), \qquad \omega \in \Omega, \quad o \in O.$$

The depth of an operand is zero, and $\text{depth}(o(e_1, e_2)) = 1 + \max\{\text{depth}(e_1), \text{depth}(e_2)\}$. Let $[\![e]\!]$ denote the value of a valid expression, and let $\nu(\cdot)$ denote the normalization used by Algorithm 2.

We prove the following induction invariant:

$$S_j = \{\nu([\![e]\!]) : e \in \mathsf{Expr}(\Omega, O),\ e \text{ is valid},\ \text{depth}(e) \le j\}.$$

Proof. For $j = 0$, Algorithm 2 initializes $S_0 = \text{normalize}(\Omega)$. These are exactly the normalized values of all valid depth-zero expressions, so the invariant holds.

Assume the invariant holds for S_{j-1}. Consider any valid expression e of depth at most j. If its depth is at most $j-1$, its normalized value already belongs to S_{j-1} and is retained in S_j.

Otherwise, $e = o(e_1, e_2)$ for some $o \in O$, where both child expressions have depth at most $j-1$. By the induction hypothesis, $\nu([\![e_1]\!])$ and $\nu([\![e_2]\!])$ occur in S_{j-1}. Algorithm 2 enumerates the pair and operation, checks validity, and adds the normalized result $\nu([\![e]\!])$ to S_j.

For the reverse inclusion, every value newly added to S_j is produced by applying an operation in O to two values represented in S_{j-1}. By the induction hypothesis, these values have expression witnesses of depth at most $j-1$; combining them yields a valid expression of depth at most j. The invariant therefore holds for S_j.

Now suppose $\hat{a}$ is generated by a valid expression e over Ω, using operations in O, with $\text{depth}(e) \le d$. The invariant gives $\nu(\hat{a}) \in S_d$. Under the audit comparator and tolerance, Algorithm 2 consequently returns $\mathsf{cert}_{\text{closure}}$. □

Witness depth.. Because closure levels are explored in increasing order of j, the first stored witness for a normalized value has minimum depth among admitted expressions that produce that value. The method therefore stores a lowest-depth witness; it need not be globally minimal under every possible expression-size measure.

Interpretation.. The proposition establishes search completeness only within the frozen operation vocabulary, normalization rules, and depth bound. It explains why a missed in-scope derivation indicates missing operands or implementation error, whereas derivations outside this bounded language may correctly receive unknown.

Appendix A.4. Proof of Proposition 1

Proof. Consider a database view containing one selected row with numeric value r. Let the question q request the average of the selected values, and let V_q require the aggregation operator AVG.

Let P_{avg} compute AVG and let P_{sum} compute SUM over the same selected row. On this database,

$$P_{\text{avg}} = r = P_{\text{sum}}.$$

Suppose the submitted program is $P^* = P_{\text{sum}}$ and the stated answer is $\hat{a} = r$. If the evidence authorization and replay conditions hold, restricted replay produces $v_{\text{replay}} = r$, so $\text{Cert}_{\text{replay}}(E) = 1$.

However, P^* implements SUM rather than the AVG operation required by V_q. Hence $S(E; V_q) =$ incorrect. We have therefore constructed an episode for which replay consistency holds while semantic correctness fails. Execution-provenance consistency consequently does not imply semantic correctness. □

The same ambiguity arises for bounded closure. For example, with $\Omega = \{4, 2\}$, both $\text{diff}(4, 2)$ and $\text{ratio}(4, 2)$ produce the value 2. A closure certificate for $\hat{a} = 2$ proves that an admitted derivation exists, but does not identify which operation was requested by q.

Interpretation.. Replay establishes the denotation of the submitted program on the current evidence view. Semantic validation establishes whether the submitted program realizes the task specification. Equal denotations on one database do not make these two relations equivalent.

Appendix B. Additional Experimental Details and Results

Appendix B.1. Frozen closure configuration

The closure configuration is selected on 60 disjoint ConvFinQA examples and then fixed for all reported evaluations. It uses $O = \{\text{sum}, \text{diff}, \text{ratio}, \text{pctchg}, \text{prod}\}$, depth $d = 2$, and relative tolerance $\varepsilon = 10^{-4}$. The same configuration is applied to the ConvFinQA remainder, MultiHiertt, and the wrong-operation analysis.

Appendix B.2. Operand-closure results

Table B.5 reports the exact counts behind RQ1. Derivation recall counts supported programs that receive closure certificates. Rejection counts aligned answer substitutions that do not receive certificates.

The standard anomaly detectors reject 53–77 of 388 substitutions (13.7–19.8%) while retaining at least 90% of supported calibration examples. Isolation Forest rejects 74/388 substitutions and retains 351/388 supported examples.

Table B.5: Exact operand-closure results.

Dataset	Method	Derivation recall	Rejection
ConvFinQA	Single operation	276/386 (0.715)	383/386 (0.992)
ConvFinQA	All cited numbers	332/386 (0.860)	240/386 (0.622)
ConvFinQA	Supervised upper	349/388 (0.899)	230/388 (0.593)
ConvFinQA	**Ledger**	**360/386 (0.933)**	**371/386 (0.961)**
MultiHiertt	Single operation	3,247/6,301 (0.515)	6,258/6,301 (0.993)
MultiHiertt	**Ledger**	**5,648/6,301 (0.896)**	**6,125/6,301 (0.972)**

Table B.6: Depth and wrong-operation analysis.

Quantity	Value
MultiHiertt recall misses with reference depth > 2	72.3%
Wrong-operation substitutions	382
Wrong-operation substitutions rejected	9/382
Substitutions within the frozen vocabulary and depth	347
In-scope substitutions rejected	0/347

Appendix B.3. Depth and wrong-operation analysis

Table B.6 records the counts used in RQ2. Most MultiHiertt misses fall beyond the frozen depth, while same-operand operation changes remain derivable inside the closure language.

Appendix B.4. Semantic-validation results

Table B.7 gives the exact WikiSQL counts. A perturbation changes aggregation, projection, predicate column or value, or predicate presence. Same-output perturbations return the same denotation as the reference query on the original database.

On the 100-case same-output sample, original-database output comparison detects 0 changes, differential execution detects 27, operator/clause signatures detect 67, and normalized SQL structure detects all 100.

Appendix B.5. Live audit and selective delivery

Table B.8 reports the complete live audit distributions. Replay denotes a replay certificate; failure denotes an observed failure record; unknown denotes an episode for which neither outcome is established.

The human-labelled KramaBench subset contains 39 clear cases: 11 are deliverable and 28 are non-deliverable. The corresponding selection outcomes appear in Table B.9.

Both policies retain all 11 human-supported cases. Replay-certificate selection removes 18 human-rejected episodes from delivery and retains 10 replayable episodes whose programs address a different quantity, scope, or operation.

The scored KramaBench subset contains 148 episodes. Replay selection delivers 95; Table B.10 reports coverage and mean score.

Appendix B.6. Runtime and model baseline

Across 500 saved records, median bookkeeping latency is 0.179 ms and the 95th percentile is 0.283 ms. The measurement covers record parsing, comparison, and closure over saved traces. Five independent runs over 100 records produce identical execution-provenance outcomes, and the audit makes zero model calls.

Table B.7: Exact WikiSQL semantic-validation results.

Quantity	Count
Unmodified reference queries	500
Controlled perturbations	2,715
Perturbations detected by normalized SQL structure	2,715/2,715
False positives on unmodified reference queries	0/500
Same-output perturbations	466
Same-output perturbations missed by output equality	466/466

Table B.8: Live execution-provenance outcomes.

Dataset	Total	Replay	Failure	Unknown
KramaBench	166	107	3	56
DABstep	60	43	0	17

Appendix C. Running-Example Audit Record

Table C.12 instantiates the audit variables for the identity-theft episode. The final program reads one national identity-theft table and 16 state-level fraud-and-other tables. It computes population as $p_i = r_i/\rho_i \times 100{,}000$, retains 14 metropolitan areas above one million population, and returns their mean report count.

Appendix D. Human Annotation and Reproducibility

The 40-row KramaBench sample spans archeology, environment, legal, and wildfire domains. Annotators receive the question, cited files, final program or program summary, stated answer, replay output, and evidence notes. The deliverability label asks whether the stated answer follows from the cited data through the recorded computation and whether that computation addresses the requested quantity. The labels are yes, no, and unclear; one unclear case is excluded from binary analysis. The benchmark answer and machine outcome are hidden during annotation.

Machine outputs are frozen before annotation. Human labels are stored separately and do not modify audit outcomes. The reproducibility package contains the frozen closure configuration, per-example arithmetic records, live trace records, semantic perturbations, baseline outputs, figure-generation code, running-example artifacts, and SHA-256 manifests. The reported rates are recoverable from the machine-readable counts listed above.

References

[1] E. Lai, G. Vitagliano, Z. Zhang, O. Chabra, S. Sudhir, A. Zeng, A. A. Zabreyko, C. Li, F. Kossmann, J. Ding, et al., Kramabench: A benchmark for ai systems on data-to-insight pipelines over data lakes, arXiv preprint arXiv:2506.06541 (2025).

[2] A. Egg, M. Iglesias Goyanes, F. Kingma, A. Mora, L. von Werra, T. Wolf, DABstep: Data agent benchmark for multi-step reasoning, arXiv:2506.23719 (2025).

[3] H. Wang, J. Liu, Y. Liu, A. S. Wijaya, T. Zhou, E. Wu, Y. Chen, W. You, R. Vir, D. Pinto, et al., Lakeqa: An

Table B.9: Selective delivery on the human-labelled KramaBench subset.

Policy	Delivered	Supported	Support rate	Escalated
Deliver all	39	11/39	28.2%	0/28
Replay certificate	21	11/21	52.4%	18/28

Table B.10: Selective delivery on scored KramaBench episodes.

Policy	Delivered	Coverage	Mean score
Deliver all	148	100.0%	0.307
Random, matched size	95	64.2%	0.307
Replay certificate	95	64.2%	0.429

Table B.11: Evidence-aware local judge results. Agreement is measured against construction labels for ConvFinQA and WikiSQL and against human deliverability labels for KramaBench. Parse failures are reported over all repeated judge calls.

Dataset	Agreement	Flip items	Parse failures
ConvFinQA	170/200	4/200	
KramaBench	25/39	1/39	29/870
WikiSQL	44/50	3/50	

Table C.12: Audit record for the running identity-theft episode.

Field	Recorded value
Question target	Mean identity-theft report count for metropolitan areas with inferred 2023 population above one million
Declared evidence	17 identity-theft CSV files
Resolved evidence	$\lvert F_{\text{res}}\rvert = 17$
Observed host reads	$\lvert F_{\text{host}}\rvert = 17$
Declared–host overlap	$\lvert F_{\text{host}} \cap F_{\text{res}}\rvert = 1$
Undeclared host reads	16 state-level fraud-and-other files
Matched metropolitan-area names	87
Rows before population filtering	85
Rows above one million population	14
Host report sum	150,138
Host output	$v_{\text{host}} = 10{,}724.1429$
Stated answer	$\hat{a} = 10{,}724.1429$
Replay status	Successful termination
Replay output	`{"main-task": null}`
Comparator result	$\text{cmp}_\varepsilon(v_{\text{replay}}, \hat{a}) = \perp$
Audit outcome	Failure record containing undeclared host reads and a missing comparable replay output
Reference recomputation	55 retained rows, report sum 713,068, and mean 12,964.8727

exploratory qa benchmark over a million-scale data lake, arXiv preprint arXiv:2606.10460 (2026).

[4] T. Gao, H. Yen, J. Yu, D. Chen, Enabling large language models to generate text with citations, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 6465–6488. doi:10.18653/v1/2023.emnlp-main.398.

[5] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, H. Hajishirzi, FActScore: Fine-grained atomic evaluation of factual precision in long form text generation, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 12076–12100. doi:10.18653/v1/2023.emnlp-main.741.

[6] T. J. Green, G. Karvounarakis, V. Tannen, Provenance semirings, in: Proceedings of the ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems, 2007, pp. 31–40. doi:10.1145/1265530.1265535.

[7] J. Cheney, L. Chiticariu, W.-C. Tan, Provenance in databases: Why, how, and where, Foundations and Trends in Databases 1 (4) (2009) 379–474. doi:10.1561/1900000006.

[8] Y. Amsterdamer, D. Deutch, V. Tannen, Provenance for aggregate queries, in: Proceedings of the ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems, 2011, pp. 153–164. doi:10.1145/1989284.1989302.

[9] L. Murta, V. Braganholo, F. Chirigati, D. Koop, J. Freire, noWorkflow: Capturing and analyzing provenance of scripts, in: Proceedings of the International Provenance and Annotation Workshop, 2014, pp. 71–83. doi:10.1007/978-3-319-16462-5_6.

[10] M. H. Namaki, A. Floratou, F. Psallidas, S. Krishnan, A. Agrawal, Y. Wu, Y. Zhu, M. Weimer, Vamsa: Automated provenance tracking in data science scripts, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2020, pp. 1542–1551. doi:10.1145/3394486.3403205.

[11] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. Maddison, T. Hashimoto, Identifying the risks of lm agents with an lm-emulated sandbox, in: International Conference on Learning Representations, Vol. 2024, 2024, pp. 27031–27098.

[12] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, F. Tramèr, Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, Advances in Neural Information Processing Systems 37 (2024) 82895–82920.

[13] A. S. Wijaya, J. Liu, H. Wang, E. Wu, SANA: What matters for QA agents over massive data lakes?, arXiv:2606.13904 (2026).

[14] R. Zhong, T. Yu, D. Klein, Semantic evaluation for text-to-SQL with distilled test suites, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, 2020, pp. 396–411. doi:10.18653/v1/2020.emnlp-main.29.

[15] F. T. Liu, K. M. Ting, Z.-H. Zhou, Isolation forest, in: Proceedings of the 8th IEEE International Conference on Data Mining, 2008, pp. 413–422. doi:10.1109/ICDM.2008.17.

[16] M. M. Breunig, H.-P. Kriegel, R. T. Ng, J. Sander, LOF: Identifying density-based local outliers, in: Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, 2000, pp. 93–104. doi:10.1145/342009.335388.

[17] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, R. C. Williamson, Estimating the support of a high-dimensional distribution, Neural Computation 13 (7) (2001) 1443–1471. doi:10.1162/089976601750264965.

[18] H. Kim, J. Taeyang, S. Choi, S. Choi, H. Cho, FLEX: Expert-level false-less execution metric for text-to-SQL benchmark, in: Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, 2025, pp. 4448–4475. doi:10.18653/v1/2025.naacl-long.228.

[19] Y. Wang, J. Zhang, T. Cai, Z. Liu, Q. Sun, Z. Sun, Z. Wu, M. Zhang, Y. Zhu, From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents, arXiv:2606.04990 (2026).

[20] Y. Nian, A. Yuan, H. Zhang, J. Li, Y. Zhao, Auditable agents, arXiv:2604.05485 (2026). doi:10.48550/arXiv.2604.05485.

[21] V. Siu, J. He, K. Montgomery, Z. Wang, N. Gong, C. Wang, D. Song, A framework for formalizing LLM agent security, arXiv:2603.19469 (2026). doi:10.48550/arXiv.2603.19469.

[22] W. Qu, M. Xu, P. Wang, S. Zhai, J. Zhang, D. Song, Securing LLM agents need intent-to-execution integrity, arXiv:2605.16976 (2026). doi:10.48550/arXiv.2605.16976.

[23] W. Hua, X. Yang, M. Jin, Z. Li, W. Cheng, R. Tang, Y. Zhang, TrustAgent: Towards safe and trustworthy LLM-based agents, in: Findings of the Association for Computational Linguistics: EMNLP 2024, 2024, pp. 10000–10016. doi:10.18653/v1/2024.findings-emnlp.585.

[24] T. Yuan, Z. He, L. Dong, Y. Wang, R. Zhao, T. Xia, L. Xu, B. Zhou, F. Li, Z. Zhang, R. Wang, G. Liu, R-Judge: Benchmarking safety risk awareness for LLM agents, arXiv:2401.10019 (2024).

[25] A. Kamath, R. Jia, P. Liang, Selective question answering under domain shift, in: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020, pp. 5684–5696. doi:10.18653/v1/2020.acl-main.503.